



如何成为智能化软件工程师？



李宣东 南京大学
2026年7月6日



南京大学计算机科学与技术系
Department of Computer Science and Technology Nanjing University



智能化时代的软件专业学习



- 为什么要成为智能化软件工程师？
- 什么是智能化软件工程师？
- 怎样才能成为智能化软件工程师？



为什么要成为智能化软件工程师？



■ 当前软件专业学生的焦虑

○ 就业市场正在大幅度萎缩、难以承载当前的人才培养规模？

- 国外工业界似乎不招初级程序员（新手）了
- 微软、谷歌、亚马逊等头部企业似乎正在大批裁人
- 人工智能浪潮打开了解决问题的空间，**但同时也侵蚀了以往软件解决问题的空间**

○ 企业对人才质量的要求大幅提高？

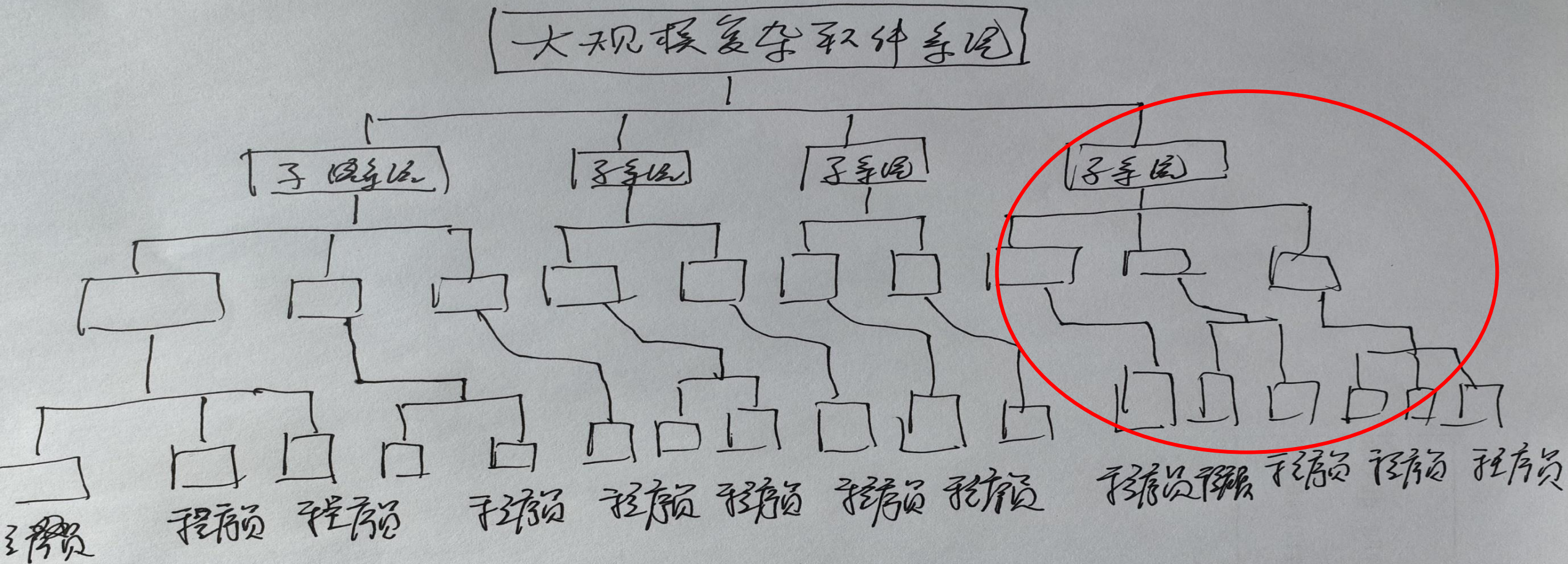
- **软件领域就业市场的红利似乎正在消失**
- 工业界不招初级程序员，但高级程序员都是从初级程序员成长起来的
- 目前工业界的期望似乎是：**学生走出高校就是高级程序员**

○ **同学们必须面对走出学校即是高级程序员的目标吗？**

- 为什么不招初级程序员？什么是高级程序员？怎样成为高级程序员？



为什么不招初级程序员？什么是高级程序员？





什么是智能化软件工程师？



■ 能够驾驭AI的高级程序员

○ 高级程序员

- 具备过硬的编程能力
- 熟练掌握软件工程原理、方法和技术
- 大规模复杂软件系统开发与演化的经历和经验

○ 驾驭AI的能力

- 驾驭原理：AI预测（生成解） + 可信性判断（验证） = （解决问题）决策
- 驾驭方式：间接驾驭，人在马下（间接引导为主）；直接驾驭，人在马上（直接引导判断）
- 驾驭目的：软件开发；软件应用



驾驭AI：原理（驾驭什么？）



■ AI解决问题的原理与过程

- 基本过程（先解后证，**解证分离**）：
 - **AI预测（生成解）** + **可信性判断（验证）** = （解决问题）决策；
- 迭代过程（解证迭代）：
 - **可信性判断（验证）** 驱动的基本过程反复迭代

■ 驾驭AI的关键问题

- **如何有效引导大语言模型生成更加准确的预测性内容（解）？**
 - 提示词、上下文工程、SDD、Agent、Harness、Agentic Engineering、...
- **如何有效针对大语言模型生成的预测性内容进行可信性判断（验证）？**
 - TDD、Agent code review、Loop Engineering、...
 - **解证分离**：解与证的难度不对称，**难解易证问题**；解与证的难度对称，**难解难证问题**
- **当前解决问题主要是运用软件工程原理、方法和技术**



驾驭AI：方式（怎样驾驭？）



■ 驾驭AI的方式

○ 间接驾驭（创造性驾驭）

- 人在马下，间接引导为主，马跑的过程**全程无人驾驭**

- 解证迭代过程**全自动**

- 氛围编程、TDD、SDD、Agents、Harness、Agentic Engineering

○ 直接驾驭（制造性驾驭）

- 人在马上，直接引导且判断，马跑的过程**全程有人驾驭**

- 解证迭代过程**人机协同**

- 基于人工开发框架，全程引入大模型加速，每一步人机达成共识（**人机融合**）





驾驭AI：方式（怎样驾驭？）



■ 软件创造与软件制造

- 软件创造：规约软件需求（分析、认识、理解和确认软件需求）
- 软件制造：根据软件需求规约构造满足需求的程序代码
- 软件创造与软件制造过程，看似分离实则相互融合
 - 规约、设计、编码、测试、调试、.....；创造1次 + 制造1次 = 1次
 - 制造未完成就不知道创造是否正确；创造不正确，制造再完美也没意义
 - 需求规约是否正确需要等代码产生后才能最终判定

■ 创造性驾驭（间接驾驭）

- 软件创造性开发（原型式开发），效率优先，在快的基础上追求更加精准

■ 制造性驾驭（直接驾驭）

- 软件制造性开发（重构式开发），质量优先，在精准的基础上追求更加高效





驾驭AI：方式（怎样驾驭？）



■ 软件创造与制造过程分离

- 软件开发过程：软件创造性开发 + 软件制造性开发；创造1次 + 制造1次 > 1次
 - 两个阶段相对分离和独立，分别对应完成软件创造任务和软件制造任务
 - 软件创造（创造性驾驭）：规约软件需求（分析、认识、理解和确认软件需求）
 - 软件制造（制造性驾驭）：根据软件需求规约构造满足需求的程序代码

■ 通过创造性驾驭是否能够完成软件制造性开发任务？

- 大模型编程，解证分离
 - AI基于概率统计原理预测生成（解），缺失世界模型因而不能自我进行可信性判断（验证）
- 人工编程，解证融合
 - 人工编程：基于程序语义和逻辑关系生成代码，解证融合
- 人在马下能达到骑在马上效果吗？！



驾驭AI：目的（为什么驾驭？）



■ 驾驭AI的目的

○ 软件开发（AI for SE）

- 软件创造性开发
- 软件制造性开发

○ 软件应用（AI in SE）

- 大语言模型是一个通用的软件系统基座

- **大语言模型 + Agents**，就是一个可以解决应用问题的软件系统，特别是解决**难解易证**问题

- 驾驭AI：针对具体应用需求，构建基于大语言模型的多智能体系统
- 类似于软件创造性开发，**但是必须认真对待可信和安全需求**





什么智能化软件工程师？



■ 能够驾驭AI的高级程序员：智能化软件工程师

○ 高级程序员

- 具备过硬的编程能力
- 熟练掌握软件工程原理、方法和技术
- 大规模复杂软件系统开发与演化的经历和经验

○ 驾驭AI的能力

- 驾驭原理：AI预测（生成解） + 可信性判断（验证） = （解决问题）决策
- 驾驭方式：间接驾驭，人在马下（解证分离）；直接驾驭，人在马上（解证融合）
- 驾驭目的：软件开发；软件应用
- 驾驭手段：软件工程原理、方法和技术



什么是智能化软件工程师？



■ 智能化软件工程师的能力需求

○ 高级程序员的能力需求

- 具备过硬的编程能力（0）
- 熟练掌握软件工程原理、方法和技术（1）
- 大规模复杂软件系统开发与演化的经历和经验（2）

○ 驾驭AI的能力需求

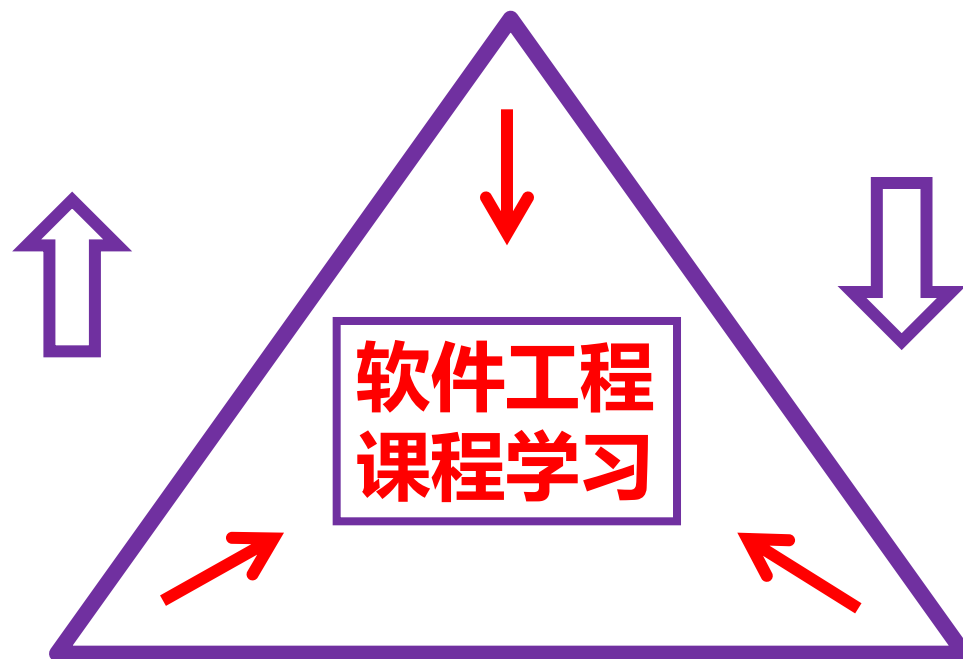
- 基于大语言模型的软件开发
 - 基于大语言模型的软件制造性开发
 - 基于大语言模型的软件创造性开发
- 基于大语言模型的软件应用
- 驾驭AI的能力核心是运用软件工程原理、方法和技术的能力（3）



怎样才能成为智能化软件工程师？



软件工程原理、方法和技术 (1)



大规模复杂系统开发实践 (2)



驾驭AI (运用软件工程原理方法技术) (3)



怎样才能成为智能化软件工程师？



- 本科学学习分为三个阶段
 - 1-2年级：严格培养手工编程能力
 - 只会编程不够，不会编程出局；不会写代码肯定不会读代码，读代码比写代码还难
 - 3年级，培养基于大模型的软件制造性开发能力
 - 4年级，培养基于大模型的软件创造性开发（软件应用）能力
- 基于大模型全面提升高年级软件工程课程的学习质量
 - 认真学习软件工程原理、方法和技术
 - 通过软件开发实践不断深刻理解软件工程原理、方法和技术
 - 基于大模型全面提升个人独立开发系统的规模和复杂性
 - 不是在软工课上学习驾驭AI，而是通过驾驭AI提升软工课程的学习质量
 - 充分理解和掌握软件工程原理和方法，具备驾驭AI独立开发较大规模系统的能力



专业学习本质是建构个性化专业世界模型



■ 通过专业学习建构自己的个性化专业世界模型

○ AI预测（生成解）+可信性判断（验证）=（解决问题）决策

■ AI基于概率统计原理预测生成（解），缺失世界模型因而不能自我进行可信性判断（验证）

■ 通过专业学习建构自己的软件专业世界模型，从而能够自信地持续学习和判断、驾驭AI

○ 人工编程：基于程序语义和逻辑关系生成代码，解证融合

○ 软件专业原来的护城河是解；AI解决问题的途径解证分离，软件专业的护城河是AI加速的解证融合

○ 软件专业世界模型

■ 基本能力：程序能力、算法能力、系统能力、工程能力

■ 综合能力（创新能力）：解决问题能力、专业知识学习能力、驾驭AI能力



问题？ ？ ？



谢谢！